

校车管理系统 —— Flutter 全栈技术方案

项目概述

本项目为校车管理系统，采用 Flutter 跨平台框架开发，覆盖三端：普通用户手机 App、司机手机 App、Web 后台管理端。

一、技术栈

层级	技术选型
前端框架	Flutter 3.x (Dart)
状态管理	Riverpod 2.x
路由	GoRouter
网络请求	Dio + Retrofit
本地存储	flutter_secure_storage (Token) / Hive (缓存)
实时通信	WebSocket (socket_io_client)
地图	flutter_map + OpenStreetMap / 高德地图 SDK
支付	微信支付 / 支付宝 SDK (pay 插件)
推送通知	Firebase Cloud Messaging (FCM)
后台管理 Web 端	Flutter Web
数据层	Repository 数据仓库层
图表	fl_chart
智能调度辅助	调度匹配算法

二、项目结构

```
school_bus_app/  
├── lib/  
│   ├── main.dart  
│   ├── app.dart  
│   └── core/  
# App 入口，路由配置
```



```
        routerConfig: buildRouter(mode), // 根据mode 加载不同路由树
        theme: AppTheme.light,
    );
}
}
```

运行命令：

用户端

```
flutter run --dart-define=APP_MODE=passenger
```

司机端

```
flutter run --dart-define=APP_MODE=driver
```

后台管理 (Web)

```
flutter run -d chrome --dart-define=APP_MODE=admin
```

四、核心功能模块设计

4.1 普通用户端

预约车次

- 展示可预约车次列表（出发地、目的地、时间、余座）
- 选座/选车次 → 确认预约 → 生成订单
- 查看我的预约（待发车 / 已完成 / 已取消）
- 取消预约（根据规则扣减信用分）

车费支付

- 订单详情页集成支付入口
- 支持微信支付 / 支付宝
- 支付状态轮询（WebSocket 推送）

信用等级

- 展示当前信用分（满分 100）
 - 失信记录明细（爽约、迟到、取消扣分规则）
 - 等级徽章：优秀 / 良好 / 一般 / 受限
-

4.2 司机端

派车任务

- 今日任务卡片列表（车次号、路线、发车时间、乘客数）

- 任务详情：乘客名单、上车点

统计看板

- 日/周/月维度切换 (fl_chart 折线图 + 柱状图)
- 统计维度：行驶里程、服务人次、准点率

实时位置共享

- 发车后自动开启 GPS 采集 (geolocator)
 - 每 5 秒通过 WebSocket 上报经纬度
 - 到站后自动停止上报
-

4.3 后台管理端 (Flutter Web)

车辆管理

- 车辆 CRUD (车牌、型号、座位数、状态)
- 车辆状态：空闲 / 运行中 / 维修中

司机管理

- 司机 CRUD (姓名、电话、驾照、绑定车辆)

车次调度

- 查看当前预约需求汇总 (出发地 → 目的地, 人数)
- **人工调度**：手动分配车辆+司机到车次
- **智能辅助调度**：调用调度算法 (贪心/最优匹配)，一键生成推荐调度方案，人工确认后生效
- 智能调度核心逻辑：

// 伪代码示例

```
List<DispatchPlan> aiDispatch(List<Booking> bookings, List<Vehicle> vehicles) {  
    // 按路线聚合预约 → 计算每路线所需车辆数 → 匹配空闲车辆 → 返回方案  
}
```

实时位置监控

- 地图展示所有在途车辆位置 (flutter_map)
- 点击车辆图标查看：车次信息、司机信息、当前速度
- 预约乘客的当前位置 (乘客端上报) 和预计到站时间 (Haversine 公式估算)

历史数据分析

- 时间范围筛选 (日/周/月/自定义)
- 图表：出行人次趋势、路线热度排行、收入统计、车辆利用率

- 数据导出 (CSV)
-

五、数据模型 (核心)

// 用户

```
class UserModel {  
    String id, name, phone;  
    int creditScore;           // 信用分 0-100  
    String role;               // passenger / driver / admin  
}
```

// 车次

```
class TripModel {  
    String id, routeName;  
    String vehicleId, driverId;  
    DateTime departureTime;  
    int totalSeats, bookedSeats;  
    TripStatus status;         // scheduled / running / completed / cancelled  
}
```

// 预约

```
class BookingModel {  
    String id, userId, tripId;  
    BookingStatus status;      // pending / paid / cancelled / completed  
    double fare;  
    DateTime createdAt;  
}
```

// 实时位置

```
class LocationUpdate {  
    String vehicleId;  
    double lat, lng;  
    double speed;  
    DateTime timestamp;  
}
```

// 司机任务统计

```
class DriverStats {  
    String driverId;  
    int totalTrips, totalPassengers;  
    double totalDistance;  
    double onTimeRate;  
}
```

六、API 接口设计 (RESTful)

```
POST    /api/auth/login
POST    /api/auth/register

GET      /api/trips                # 获取可预约车次列表
POST     /api/bookings            # 创建预约
DELETE   /api/bookings/:id        # 取消预约
GET      /api/bookings/mine        # 我的预约

POST     /api/payments            # 发起支付

GET      /api/driver/tasks         # 司机派车任务
GET      /api/driver/stats         # 司机统计数据
WS       /ws/location             # WebSocket: 位置上报/订阅

GET      /api/admin/vehicles       # 车辆列表
POST     /api/admin/vehicles       # 新增车辆
PUT      /api/admin/vehicles/:id   # 编辑车辆
DELETE   /api/admin/vehicles/:id   # 删除车辆

GET      /api/admin/dispatch/suggest # 智能辅助调度建议
POST     /api/admin/dispatch/confirm # 确认调度方案

GET      /api/admin/analytics      # 历史数据分析
```

七、WebSocket 实时通信

```
// 位置上报 (司机端)
socket.emit('driver:location', {
  'vehicleId': vehicleId,
  'lat': position.latitude,
  'lng': position.longitude,
  'timestamp': DateTime.now().toIso8601String(),
});

// 后台订阅所有车辆位置
socket.on('vehicles:locationUpdate', (data) {
  ref.read(vehicleLocationsProvider.notifier).update(data);
});
```

八、快速启动

- 1. 安装依赖
flutter pub get
- 2. 按需选择入口启动普通用户端、司机端或后台管理端。

九、关键第三方包清单（pubspec.yaml）

```
dependencies:  
  flutter_riverpod: ^2.5.1  
  go_router: ^14.0.0  
  dio: ^5.4.3  
  hive_flutter: ^1.1.0  
  flutter_secure_storage: ^9.0.0  
  socket_io_client: ^2.0.3+1  
  geolocator: ^12.0.0  
  flutter_map: ^7.0.0  
  latlong2: ^0.9.0  
  fl_chart: ^0.68.0  
  pay: ^2.0.0  
  firebase_messaging: ^15.0.0  
  json_annotation: ^4.9.0  
  freezed_annotation: ^2.4.1  
  intl: ^0.19.0  
  
dev_dependencies:  
  build_runner: ^2.4.9  
  json_serializable: ^6.8.0  
  freezed: ^2.5.2
```

十、开发优先级（建议交付顺序）

阶段	内容	预计工时
P0	登录/注册、路由骨架、演示数据	1 天
P1	用户端：预约、查看、取消	1 天
P2	司机端：任务列表、位置上报	1 天
P3	后台：车辆/司机管理、实时地图	1.5 天

阶段	内容	预计工时
P4	后台：调度（含智能建议）、数据分析	1 天
P5	支付、信用分、推送通知、UI 打磨	1 天

本方案基于 *Flutter 3.x + Riverpod + GoRouter*，采用功能模块化目录结构，三端通过 *APP_MODE* 环境变量切换，共用核心数据层和网络层，最大化代码复用。